

“Online Banking System Simulation”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

JAKKA BHANU

:2520030006

ANAND AKSHITA

:2520030043

Under the Esteemed Guidance of

Dr. D. Ramya Krishna

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **“Online Banking System Simulation”** is a record of Bonafide work of **JAKKA BHANU - 2520030006, ANAND AKSHITA - 2520030043** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

JAKKA BHANU – 2520030006

ANAND AKSHITA – 2520030043

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **“Online Banking System Simulation”** is a bonafide work done and submitted by **JAKKA BHANU - 2520030006, ANAND AKSHITA - 2520030043** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in Department of Computer Science and Engineering, K L (Deemed to be University)**, during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator and our guide **Dr. D. Ramya Krishna**, Department of Computer Science and Engineering, which cannot be expressed in words, for the tremendous support, encouragement and guidance given for this project.

We express our gratitude to the Head of the Department, **Department of Computer Science and Engineering**, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of teaching and non-teaching staff members, and also those who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

JAKKA BHANU – 2520030006

ANAND AKSHITA – 2520030043

Abstract

Modern banking depends on correct and consistent data. Even a small error in a balance, a transfer that is applied only half-way, or two customers updating the same account at the same time can cause real financial loss and a loss of trust. Many simple banking applications built as academic exercises ignore these problems: they mix business logic with user-interface code, store redundant data, do not control concurrent access, and become slow as the number of records grows. To tackle this, our project focuses on the design and development of an Online Banking System Simulation. Our core aim was to build a full-stack banking application whose database design and transaction handling remain reliable under realistic use.

The system, presented to the user under the name Vantage Bank, supports the essential banking operations: creating and viewing customer accounts, deposits, withdrawals, fund transfers between accounts, and a complete transaction history. The back end is built with Spring Boot and exposes REST APIs, the front end is a React.js application with Login, Dashboard, Transfer, Transaction History, Cards and Profile screens, and the data is stored in a MySQL database made up of four related tables: customers, accounts, cards and transactions.

The most important part of the project is its database engineering. The schema is normalised to Boyce-Codd Normal Form (BCNF) so that every fact is stored exactly once. Every money movement runs inside an ACID transaction with suitable isolation levels, so that a transfer either debits and credits both accounts or changes nothing. Indexes are added on frequently searched columns and verified with the MySQL EXPLAIN statement to keep queries fast as data grows, and the project also studies replication and sharding as ways of scaling a banking database beyond a single server. The APIs were tested with Postman, and account balances were found to remain consistent after transfers. In summary, this project delivers a working, layered and reliable banking simulation that meets all of its stated objectives, and it provides a foundation for future work such as JWT-based login security and real replication and sharding.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-8
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	9-10
3	Technology Stack	3.1 Front-end design (React.js) 3.2 Back-end (Spring Boot, REST APIs) 3.3 Database (MySQL) 3.4 Version control (Git, GitHub) 3.5 APIs and tools (Postman, MySQL Workbench)	11-13
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture	14-15
5	Design	5.2 ER Diagram / Database schema	16
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization 6.5 Transaction handling	17-19
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	20-21
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	22-23
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	24-25
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	26-27
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	28

S. No.	Chapters	Topics	Page.no
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	29
13	References	- Books, tutorials, APIs, documentation sites used	30
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures	31-35

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Banking is one of the most important services in any economy. Individuals use banks to save money, receive salaries and pay for goods and services, while businesses depend on them for payments, credit and record keeping. In earlier decades every one of these actions needed a visit to a branch, a paper form and a manual entry in a ledger. Today most of these operations are performed through online and mobile channels, and a customer expects to deposit, withdraw, transfer money and check a statement within a few seconds, at any time of day.

Behind every such screen there is a database that must always be right. A bank does not only store data; it stores a promise about money. If a customer transfers an amount from one account to another, the amount must leave the first account and arrive in the second, exactly once, even if the server fails half-way or many other customers are using the system at the same moment. For this reason, banking applications are a classic case study in database systems engineering, where the correctness of the data model and of transaction handling matters more than the appearance of the interface.

The idea for this project came from studying how small academic banking applications are usually built. They often place SQL queries inside user-interface code, keep customer details repeated in several tables, update balances with separate statements that are not grouped into a transaction, and test only with a single user. Such systems appear to work in a demonstration but fail when two requests arrive together or when the number of records grows. The Online Banking System Simulation was proposed to show how these weaknesses can be removed by applying the concepts studied in the course Database Systems Engineering and Distributed Backend Development.

Technological growth in web development frameworks has made it practical to build such a system with open-source tools. Spring Boot provides a structured way to build REST APIs in layers, React.js allows a responsive single-page interface, and MySQL provides a mature relational engine with support for ACID transactions, row-level locking, indexing and the EXPLAIN statement for analysing query plans. Together they allow a student team to build a system that follows the same architectural ideas as real banking back ends, at a small scale.

From an academic and engineering perspective, the background of the project lies in combining several core areas of computer science:

- Database Design: converting requirements into entities and relationships and normalising them to BCNF to remove redundancy and update anomalies.
- Transaction Processing: applying the ACID properties and isolation levels so that concurrent operations never corrupt balances.

- Query Performance: using indexes and the EXPLAIN statement to reduce the cost of searches on growing tables.
- Distributed Data Concepts: studying replication and sharding as the way a single database is scaled and made fault-tolerant.
- Back-End Engineering: building a layered REST API in which controllers, validation, services and repositories have separate responsibilities.
- User Experience: giving the customer a simple dashboard, transfer form and history view.

To summarize, the background of this project rests on three fundamental observations:

- Banking data must be consistent at all times, and consistency is a property of the database design, not only of the application code.
- Simple banking systems commonly suffer from redundant data, uncontrolled concurrency and slow queries.
- Open-source technologies now make it possible to build, test and study a reliable layered banking back end within a single academic trimester.

1.2 Problem Statement

Simple banking systems have inconsistent data, concurrency errors and slow queries. This single sentence summarises three connected problems that this project addresses.

The first problem is inconsistent data. When customer details, account details and transaction details are stored together in wide tables, the same fact is repeated in many rows. A change of address or a corrected name must then be applied in several places, and if one of them is missed the database contradicts itself. Balances are even more sensitive: if the balance is stored in one place and the sum of transactions implies another value, the system can no longer say how much money a customer really has.

The second problem is concurrency. A banking system is used by many customers at once, and sometimes by the same customer from two devices. If two withdrawals read the same balance before either has written the new one, both may succeed and the account can go below zero, or one update can silently overwrite the other. Without transactions and a clear isolation policy, a transfer that fails after the debit but before the credit leaves money missing from the system.

The third problem is performance. A history page or an account search that scans every row of a large table is acceptable for ten records and unacceptable for ten million. Systems built without indexes, or without checking query plans, become slow as the data grows, and the slowness is discovered only after the system is already in use. Finally, a single database server is both a performance limit and a single point of failure, which raises the question of how replication and sharding can be used.

By addressing these fundamental issues, the project aims to achieve the following problem resolutions:

- Eliminate redundancy and update anomalies through a BCNF-normalised schema.
- Guarantee safe money movement through ACID transactions with appropriate isolation levels.
- Keep queries fast as data grows through indexing verified by EXPLAIN.
- Separate responsibilities through a layered architecture of controller, validation, service and repository.
- Study how replication and sharding overcome the limits of a single database.

1.3 Scope of the Project

The scope of the project is limited to the core of a retail banking system. The following areas are included in the implementation: Customers, accounts, transfers and history. In practical terms this means that the system covers customer records, the opening and viewing of savings and current accounts, deposits, withdrawals, transfers between accounts, a record of every transaction, and the storage of card details for each account.

The project is a simulation. It does not connect to a real payment network, does not move real money and does not implement regulatory features such as loans, interest calculation, cheque clearing or foreign exchange. The concepts of replication and sharding are studied and explained, and their application to the banking schema is discussed, but a multi-server deployment is outside the implemented scope and is listed under future enhancements.

1.4 Objective(s)

The primary objective of the project is to build a reliable full-stack banking simulation that demonstrates sound database engineering. This is broken into three specific objectives:

- Objective 1: Design a BCNF-normalised schema. The data must be organised into customers, accounts, cards and transactions so that no fact is stored twice and no update anomaly can occur.
- Objective 2: Ensure safe transactions with ACID. Deposits, withdrawals and transfers must be atomic, must keep the data consistent, must be isolated from concurrent requests and must be durable once committed.
- Objective 3: Improve speed with indexing and study replication/sharding. Searches and history queries must be supported by indexes whose effect is checked with EXPLAIN, and the project must study how replication and sharding would be used to scale the system.

Secondary objectives are to follow a layered code structure that is easy to maintain, to test every API with Postman, to manage the source code with Git and GitHub, and to provide a simple and clear interface for the customer.

1.5 Importance of the Application

The importance of the application lies in showing, in a working system, ideas that are normally met only as theory. Normalisation, transactions and indexing are often taught with small tables on paper; in this project the same ideas decide whether a balance is correct, which makes their value visible and measurable.

For the user of the system, the importance is dependability. A customer can deposit, withdraw and transfer money, review the transaction history and see the balance of every account on one dashboard, and can be sure that the figures agree. For developers and students, the importance is educational: the project is a compact reference showing how a layered Spring Boot application is connected to a relational database through a transaction boundary, and how design decisions in the schema influence performance later.

The project also has an academic and research importance. It provides a base on which security (JWT authentication), distributed data (real replication and sharding) and analytics can be added in later phases without redesigning the core.

1.6 Target Users/Audience

The primary users of the system are bank customers who need to manage their accounts online. They are expected to open an account, check balances, deposit and withdraw funds, transfer money to another account and review their history. The interface is therefore kept simple, with a small number of clearly named screens.

The secondary users are students, teachers and developers who study the project as a learning resource for database systems engineering and distributed back-end development. The documentation, ER diagram, architecture diagram and test cases are intended to help them understand how a reliable banking back end is structured.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements form the foundation of any software project. They define the physical resources needed to develop, test and deploy the application. Since the Online Banking System Simulation is built on standard, open-source tools, it does not need specialised or expensive equipment; an ordinary development laptop is sufficient. The hardware requirements are divided into those needed by developers during coding and testing, and those needed by the machine that hosts the application.

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core Intel Core i5 / AMD Ryzen 5 or higher
Memory (RAM)	8 GB	16 GB, so that the IDE, MySQL server, Node.js and browser run together smoothly
Storage	20 GB free	256 GB SSD for faster builds and database access
Display	1366 x 768	Full HD (1920 x 1080) for working with several tools side by side
Network	Broadband connection	Stable connection for downloading dependencies and using GitHub
Peripherals	Keyboard, mouse	Additional monitor and headphones for collaboration

A multi-core processor and at least 8 GB of RAM are important because the development workflow runs several heavy programs at the same time: the Spring Boot application, the React development server, the MySQL server and its Workbench client, the Postman application and a web browser. A solid-state drive reduces the time needed to build the project and to load large database files. The MySQL server itself is light and runs comfortably on this hardware, although query behaviour on larger data volumes benefits from additional memory for the buffer pool.

2.2 Software Requirements

The software requirements describe the operating system, runtime environments and applications that must be installed to build and run the system. Every item in the list is free or open source, which supports the economic feasibility of the project.

Category	Software	Purpose
Operating system	Windows 10/11, Ubuntu Linux or macOS	Development and execution platform
Runtime	Java Development Kit (JDK) 17 or higher	Compiles and runs the Spring Boot back end

Category	Software	Purpose
Build tool	Maven	Dependency management and project build
Runtime	Node.js and npm	Runs the React development server and installs packages
Database	MySQL Server 8.x	Stores customers, accounts, cards and transactions
Database client	MySQL Workbench	Schema design, SQL execution and EXPLAIN analysis
API testing	Postman	Sending requests and verifying responses of the REST APIs
Version control	Git and GitHub	Source management and collaboration
Browser	Google Chrome / Microsoft Edge / Firefox	Running the React front end

2.3 Development Tools and Frameworks

A development environment is a combination of editors, frameworks and utilities. For this project the following tools were chosen because they are standard, well documented and widely used in industry, which also makes problems easy to research and solve.

- IDE and editors: IntelliJ IDEA or Eclipse for the Spring Boot code, and Visual Studio Code for the React code.
- Spring Boot: the framework used to build the REST API with embedded server, dependency injection and data-access support.
- React.js: the library used to build the component-based user interface.
- MySQL Workbench: used for modelling the ER diagram, running SQL scripts and studying query plans with EXPLAIN.
- Postman: used to create requests for every endpoint, to send valid and invalid inputs and to check status codes and response bodies.
- Git and GitHub: used for version control, history of changes and sharing of code between the team members.

Using one coherent set of tools reduces configuration problems. The team members work on the same code base through GitHub, run the same database scripts and test the same Postman requests, so a result obtained on one machine can be reproduced on the other.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End Design

The front end of the Online Banking System Simulation is the interface through which a customer interacts with the bank. It is developed in React.js, a JavaScript library for building user interfaces from small reusable components. A banking interface is a good fit for this approach because the same elements, such as an account card, a transaction row or a form field, appear many times on different screens.

The interface is organised as a single-page application with the following screens, which are also listed in the navigation menu of the dashboard:

- Login: the entry point where the customer identifies himself or herself.
- Dashboard: shows a welcome message, the total balance, each account with its type, status and masked account number, and the most recent transactions, together with an Open new account button.
- Transfer: a form for moving money from one of the customer's accounts to another account.
- Transaction History: a complete, ordered list of past transactions.
- Cards: the cards linked to the customer's accounts, displayed with masked numbers.
- Profile: the customer's personal details.

React keeps the state of each screen in components and updates only the parts of the page that change when data arrives, which gives a smooth experience without reloading the page. The front end communicates with the back end only through REST APIs that exchange JSON, so the interface contains no banking rules. This is an important design decision: if the interface were to calculate balances, a modified browser could change them. Therefore the interface presents data and collects input, while every rule is enforced on the server.

Amounts are shown in Indian rupees with credits and debits clearly distinguished by sign and colour, and transaction types such as Transfer, Deposit and Withdrawal are displayed as labelled tags so that a customer can scan the history quickly. The layout is kept simple, with a dark side navigation panel and light content area, so that it remains easy to use for customers with different levels of technical experience.

3.2 Back-End (Spring Boot)

The back end is built with Spring Boot and exposes the banking functions as REST APIs. Spring Boot was selected because it removes much of the configuration work from Java web development: it provides an embedded server, automatic configuration, dependency injection and strong support for database access and transaction management.

The back end follows a layered architecture in which each layer has a single responsibility:

- **REST Controller:** receives HTTP requests, routes them to the correct operation and returns JSON responses with suitable status codes.
- **Validation:** checks the input, confirms that the account belongs to the requesting customer (ownership) and confirms that the balance is sufficient for the operation.
- **Service Layer:** holds the banking rules, such as how a transfer debits one account and credits another, and defines the transaction boundary for each operation.
- **Repository:** performs the SQL operations against the database.

Because the layers are separate, a change in one of them does not force changes in the others. A new validation rule can be added without touching the repository, and the database access code can be optimised without changing the controllers. This structure is also the reason the proposed system is described as having a layered architecture compared with the mixed logic of simple banking systems.

3.3 Database (MySQL)

MySQL is the relational database management system used to store all banking data. It was chosen because it is open source, widely documented and supports the features on which this project depends: the InnoDB storage engine with ACID transactions, row-level locking, configurable isolation levels, B-tree indexes, foreign key constraints and the EXPLAIN statement.

The database contains four tables: customers, accounts, cards and transactions. A customer can own many accounts, an account can have cards, and an account can have many transactions. The tables are normalised to BCNF, which means that every determinant in a table is a candidate key, and thus no non-key attribute depends on anything other than the key. The full design is described in Chapter 5.

Transactional behaviour is central to the choice of engine. InnoDB writes changes to a redo log before they are acknowledged, so a committed transfer survives a crash (durability), and it can undo an incomplete transfer through the undo log (atomicity). Row-level locks allow different customers to work on different accounts at the same time, while conflicting operations on the same account are serialised.

Query performance is addressed through indexes. Primary keys are indexed automatically, and secondary indexes are added on columns used in search conditions, such as the account identifier and the creation time in the transactions table, so that a customer's history can be retrieved without scanning the entire table. The effect of each index is checked by running EXPLAIN before and after the index is created.

3.4 Version Control (Git and GitHub)

Git is used for version control and GitHub for hosting the repository. Every change to the source code and database scripts is committed with a message that explains it, so that the history of the project is traceable and any faulty change can be reverted. Both team members work on the same repository, and the use of separate branches for separate features reduces the risk of overwriting each other's work.

GitHub also serves as the shared location of the finished project; the QR code printed on the project poster links to the repository, so that evaluators can inspect the source code, the SQL scripts and the documentation.

3.5 APIs and Tools

The system exposes its own REST APIs rather than depending on external third-party services, which keeps the project free of cost and fully under the control of the team. The principal groups of APIs are the following:

API group	Operation	Description
Customers	Create / view customer	Registers and retrieves customer details
Accounts	Open / view account, view balance	Creates savings or current accounts and returns balances
Transactions	Deposit, withdraw, transfer	Applies money movements inside an ACID transaction
History	List transactions of an account	Returns ordered transaction records for display
Cards	View cards of an account	Returns masked card details and status

Postman is the main tool for working with these APIs. Each endpoint has a saved request with example inputs, and requests were grouped into collections so that the entire test suite can be repeated after every change. MySQL Workbench complements Postman at the database level, since it is used to inspect the stored rows after an API call and to study query plans.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

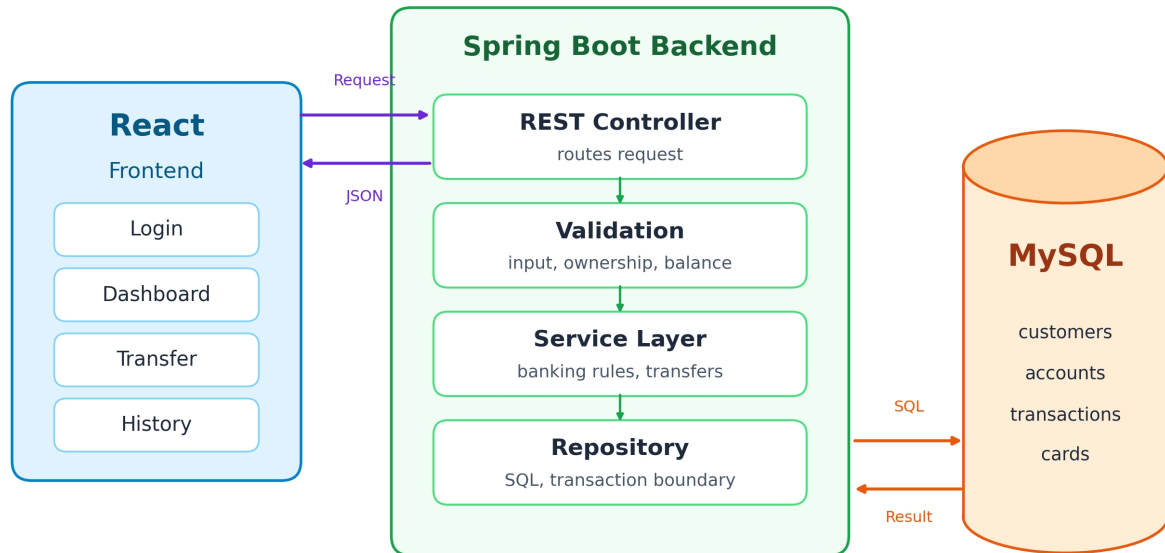


Figure 4.1: High-level architecture of the Online Banking System Simulation (React front end, Spring Boot back end, MySQL database)

4.2 Description of Each Layer (Frontend, Backend, Database)

The Online Banking System Simulation follows a three-tier architecture in which the front end, the back end and the database are separated and communicate only through well-defined interfaces. Figure 4.1 shows how a request travels from the customer's browser through the layers of the back end to the database and back.

Front-end layer. The React front end runs in the customer's browser and contains the Login, Dashboard, Transfer and History screens. It sends a request to the back end for each action, such as loading the dashboard or submitting a transfer, and receives the answer as JSON. It performs only basic input checks for the convenience of the user; it does not hold any banking rule.

Back-end layer. The Spring Boot back end is itself divided into four layers, as shown in the diagram. The REST Controller receives the request and routes it. The Validation layer checks the input, the ownership of the account and the sufficiency of the balance, and rejects invalid requests before any change is made. The Service Layer applies the banking rules, for example the sequence of debiting one account, crediting another and recording the transaction, and it marks the beginning and end of the database transaction. The Repository executes the SQL statements and is therefore the only part of the code that talks to the database.

Database layer. MySQL stores the four tables customers, accounts, transactions and cards. It receives SQL from the repository, executes it inside the transaction boundary set by the service layer and returns the result. Because the ACID guarantees are provided by the database engine, the correctness of balances does not depend on every line of application code being perfect.

Layer	Technology	Responsibility
Presentation	React.js	Screens, forms, display of balances and history
REST Controller	Spring Boot	Receives requests and routes them; returns JSON
Validation	Spring Boot	Input, ownership and balance checks
Service	Spring Boot	Banking rules and transaction boundary
Repository	Spring Boot / SQL	Reads and writes data through SQL
Data	MySQL	Persistent, consistent storage with indexes

Request flow. For a transfer, the customer fills the Transfer form and the browser sends a request to the controller. The validation layer checks that the source account belongs to the customer, that the destination exists and that the balance covers the amount. The service layer then starts a transaction, reduces the balance of the source account, increases the balance of the destination account, inserts the transaction records and commits. If any step fails, the whole transaction is rolled back and an error response is returned. The front end finally refreshes the dashboard to show the new balances.

4.3 Deployment Architecture

The simulation is deployed locally. The React application runs on the Node.js development server, the Spring Boot application runs on its embedded server and the MySQL server runs on the same machine or on another machine in the local network. The three components communicate over HTTP and the MySQL protocol using configurable host names and ports, so they can be moved to separate machines without code changes.

Source code is stored in a GitHub repository, which acts as the single source of truth for the project. Because the project is a simulation built with open-source tools, no paid cloud service is required. In future work, the same structure can be placed behind a web server and the database can be replicated across several servers, as discussed in Chapter 11.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database Schema

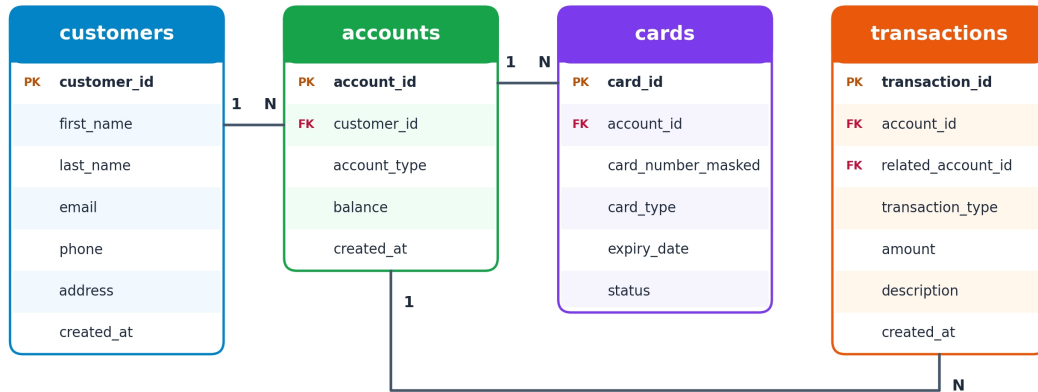


Figure 5.1: ER diagram and database design of the Online Banking System Simulation

Figure 5.1 shows the entity-relationship design. The schema has four entities. A customer can own many accounts (1 : N), an account can have many cards (1 : N), and an account can have many transactions (1 : N). The transactions table also holds a second reference, `related_account_id`, to the other account that takes part in a transfer.

Table	Attributes	Keys
customers	customer_id, first_name, last_name, email, phone, address, created_at	PK: customer_id
accounts	account_id, customer_id, account_type, balance, created_at	PK: account_id; FK: customer_id
cards	card_id, account_id, card_number_masked, card_type, expiry_date, status	PK: card_id; FK: account_id
transactions	transaction_id, account_id, related_account_id, transaction_type, amount, description, created_at	PK: transaction_id; FK: account_id, related_account_id

Normalisation. Every table is in BCNF: in each table the only determinants are candidate keys. Customer details are stored once in customers and referenced by `customer_id` from accounts, so changing an address is a single update. Card data depends on the card, not on the customer, so it is kept in its own table. A transaction stores the account identifiers rather than copies of account or customer details, which removes redundancy and prevents update anomalies.

Indexing. Primary keys are indexed automatically. In addition, secondary indexes are planned on the columns that are used for searching: `transactions(account_id, created_at)` for the history query, `accounts(customer_id)` for loading the accounts of a customer and `customers(email)` for lookups by email. EXPLAIN confirms that these queries use the indexes.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The Online Banking System Simulation is implemented as a set of cooperating modules. Each module has its own screens in the front end, its own endpoints in the back end and its own tables in the database, but all modules share the same layered structure.

Customer and Account module. This module handles the creation of customers and the opening and viewing of accounts. A customer can hold more than one account, for example a savings account and a current account, and the dashboard shows each of them with its balance, type, status and masked number, along with the total balance. Opening a new account inserts a row in the accounts table linked to the customer through `customer_id`.

Deposit and Withdrawal module. A deposit increases the balance of an account and a withdrawal decreases it. Both are executed inside a transaction: the service layer loads the account, checks the rules, updates the balance and inserts a row in the transactions table in one unit. For a withdrawal, the validation layer first confirms that the balance is sufficient, so an account cannot be overdrawn.

Transfer module. A transfer moves money between two accounts. It is the most sensitive operation of the system, because it changes two rows and must either change both or neither. The service layer runs the entire operation in a single transaction: it locks the two account rows, checks the balance, debits the source, credits the destination and records the transaction with the `related_account_id`. Any failure causes a rollback.

Transaction History module. This module returns the transactions of an account in order of date. Each row carries a description, the date, the type (Transfer, Deposit or Withdrawal) and the amount, and the front end shows credits and debits with different colours. The query is supported by an index on the account identifier and the creation time.

Cards module. This module lists the cards linked to an account, showing the masked card number, type, expiry date and status.

6.2 Front-End Logic (UI Rendering, State Management)

The React front end is built from components. A page-level component for each screen fetches the data it needs when it is displayed and stores it in component state; the lower-level components, such as an account card or a transaction row, receive the data as properties and only render it. When an action completes, for instance when a transfer succeeds, the screen reloads the affected data so that the new balances and the new history entry appear without a full page reload.

Forms validate basic input, such as a missing field or a non-numeric amount, before sending a request, and the server responses are used to show clear messages if the operation is rejected. Navigation between the Dashboard, Transfer, Transaction History, Cards and Profile screens is performed from the side menu without reloading the

page.

6.3 API Integration

The front end and back end are integrated through REST APIs that exchange JSON. The standard HTTP methods are used according to their meaning: GET to read data, POST to create a resource or perform an operation such as a transfer, PUT or PATCH to update, and DELETE to remove. The server uses HTTP status codes to describe the outcome, for example 200 or 201 for success, 400 for invalid input, 404 for a missing account and 409 or 422 for a business-rule violation such as insufficient balance.

Purpose	Method	Endpoint (illustrative)
Create customer	POST	/api/customers
Open account	POST	/api/accounts
View accounts and balances	GET	/api/customers/{id}/accounts
Deposit	POST	/api/accounts/{id}/deposit
Withdraw	POST	/api/accounts/{id}/withdraw
Transfer	POST	/api/transfers
Transaction history	GET	/api/accounts/{id}/transactions
View cards	GET	/api/accounts/{id}/cards

The endpoint names above are shown to illustrate the structure of the API; the exact paths are defined in the controllers of the source code in the GitHub repository.

6.4 Authentication & Authorization

The Login screen is the entry point of the application. Authorization in the current version is enforced by the Validation layer of the back end, which checks the ownership of every account involved in a request: a customer can operate only on his or her own accounts, and a request that names another customer's account is rejected before any change is made. The balance check in the same layer prevents overdrafts.

Token-based authentication using JSON Web Tokens (JWT) is not part of the present implementation and has been planned as the first item of future scope. With JWT, the server would issue a signed token at login and every later request would carry it, so the identity of the customer would be verified on each call.

6.5 Transaction Handling

The core of the implementation is the way transactions are controlled. The service layer marks the boundary of each operation, so that all the statements of a transfer belong to one database transaction. A simplified outline of the transfer operation is shown below.

```

BEGIN;
-- lock both rows in a fixed order to avoid deadlocks
SELECT balance FROM accounts WHERE account_id IN (:from, :to)
    ORDER BY account_id FOR UPDATE;
-- (service layer verifies balance >= :amount)
UPDATE accounts SET balance = balance - :amount WHERE account_id = :from;
UPDATE accounts SET balance = balance + :amount WHERE account_id = :to;
INSERT INTO transactions (account_id, related_account_id,
    transaction_type, amount, description)
VALUES (:from, :to, 'TRANSFER', :amount, :note);
COMMIT;  -- any error => ROLLBACK

```

The four ACID properties are provided as follows. Atomicity: the debit, the credit and the history record succeed or fail together. Consistency: constraints and the balance check keep the data valid before and after the operation. Isolation: row locks and the chosen isolation level prevent two concurrent transfers from reading and overwriting the same balance. Durability: once the transaction is committed, InnoDB's logs ensure that it survives a server restart.

Locking the rows in a fixed order, as in the outline above, is a standard technique to avoid deadlocks, in which two transfers going in opposite directions would otherwise wait for each other.

CHAPTER -7 FEATURES

7.1 List of Main Features

The Online Banking System Simulation offers the following main features, each of which is explained from the user's perspective and from the technical perspective in Section 7.2.

- Customer dashboard with a welcome message, total balance and account summary.
- Opening and viewing of savings and current accounts.
- Deposits and withdrawals with automatic balance update.
- Transfers between accounts with all-or-nothing behaviour.
- Transaction history with date, type, description and amount.
- Card listing with masked card numbers and status.
- BCNF-normalised database design.
- ACID transactions with isolation levels for concurrent safety.
- Indexing and EXPLAIN-based query analysis for performance.
- Layered REST architecture that is easy to test and maintain.

7.2 How Each Feature Works (User Perspective + Technical Description)

Dashboard. User perspective: after logging in, the customer sees a welcome message, the total balance, one card for each account showing the account type, status and masked number, and a list of recent transactions, together with a button to open a new account. Technical description: the screen calls the accounts and history APIs; the total balance is the sum of the balances of the customer's accounts, and the recent transactions are the latest rows from the transactions table, retrieved through the index on account and time.

Deposit and withdrawal. User perspective: the customer enters an amount and the balance changes at once; a withdrawal larger than the balance is refused with a clear message. Technical description: the request passes through the validation layer, and the service layer updates the balance and inserts a transaction row inside one database transaction. The transaction type is stored as Deposit or Withdrawal.

Transfer. User perspective: the customer chooses the account to debit, the destination account and the amount; the transfer either completes or is rejected, and the history shows the entry as a debit for the sender and a credit for the receiver. Technical description: both accounts are locked, checked and updated in one transaction, and the related_account_id column records the counterpart account. If an error occurs, the rollback restores both balances.

Transaction history. User perspective: the customer can review past activity in order, with the type of each entry shown as a tag and the amount coloured to separate credits from debits. Technical description: a query on transactions filtered by account_id and ordered by created_at uses a composite index; EXPLAIN confirms the use of the index and a small number of examined rows.

Cards. User perspective: the customer can see the cards linked to an account, with only the last digits of the number visible and the expiry date and status shown. Technical description: the cards table stores only the masked number, which reduces the risk of exposing sensitive data.

Reliability features. The BCNF schema, the ACID transactions and the indexes are not visible on any screen, but they are the features on which the visible ones depend. They ensure that the balance displayed in the Dashboard agrees with the transactions listed in the History and remains correct when several requests arrive together.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was used to test the REST APIs of the back end independently of the user interface. For every endpoint a request was saved with a sample body, and the requests were organised into collections for customers, accounts, deposits and withdrawals, transfers, history and cards. Testing the APIs directly makes it possible to check the back end rules without any influence from the front end, and a failure can be traced directly to the layer that caused it.

Each request was checked for the HTTP status code, the structure of the JSON response and the effect on the database. After a successful call, MySQL Workbench was used to confirm that the rows in the accounts and transactions tables had changed as expected. Invalid input, such as a negative amount, an unknown account or an amount larger than the balance, was also sent to confirm that the validation layer rejects it with a suitable error and leaves the data unchanged.

The most important check concerned transfers. After transfers were executed, the balances of the two accounts were compared with their transaction records, and the balances were found to stay consistent: the amount subtracted from the source equals the amount added to the destination, and the history agrees with the balances.

8.2 Integration Testing

Integration testing verifies that the front end, back end and database operate correctly as a whole. Starting from the Login screen, the main flows were exercised through the interface: viewing the dashboard, making a deposit, making a withdrawal, making a transfer and checking the history. In each flow, the figures displayed by the interface were compared with the values stored in the database, to confirm that the three tiers interpret and present the data in the same way.

8.3 Tools Used for Testing

Tool	Use
Postman	Sending requests to the REST APIs and checking status codes and response bodies
MySQL Workbench	Inspecting rows after each operation; running EXPLAIN on queries
Web browser	Manual testing of the React screens
Git and GitHub	Keeping the test collections and scripts under version control

8.4 Test Cases and Results

The test cases below cover the main functions and the main failure conditions of the system. The status column records whether the result has been confirmed by the project team.

ID	Test case	Expected result	Status
TC-01	Create a customer with valid details	Customer is stored and its identifier returned	Pass
TC-02	Open a savings account for a customer	Account is created and linked to the customer	Pass
TC-03	Deposit a valid amount	Balance increases; a Deposit transaction is recorded	Pass
TC-04	Withdraw an amount within the balance	Balance decreases; a Withdrawal transaction is recorded	Pass
TC-05	Withdraw more than the balance	Request rejected; balance unchanged	To verify
TC-06	Transfer between two accounts	Source debited and destination credited by the same amount	Pass
TC-07	Transfer with insufficient balance	Request rejected; neither balance changes	To verify
TC-08	Transfer fails after the debit step	Rollback restores both balances	To verify
TC-09	Two transfers on the same account at once	Balances remain correct; no lost update	To verify
TC-10	View transaction history of an account	Transactions are listed with type, date and amount	Pass
TC-11	Balance consistency after transfers	Balances agree with the transaction records	Pass
TC-12	Access an account of another customer	Request rejected by ownership validation	To verify

Summary of results. The APIs were tested in Postman and the balances stayed consistent after transfers. The remaining cases marked To verify describe the failure and concurrency conditions that the design is intended to handle through validation, rollback and isolation; their outcomes are to be confirmed and recorded by the team before final submission.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The Online Banking System Simulation is deployed on a local machine using the open-source tools listed in Chapter 2. The deployment follows a fixed order, because the back end needs the database to be ready and the front end needs the back end to be reachable.

- Install the prerequisites: JDK 17 or higher, Maven, Node.js with npm, and MySQL Server 8.x together with MySQL Workbench.
- Clone the project from the GitHub repository (the repository link is available through the QR code on the project poster).
- Create the database in MySQL and run the SQL script that creates the customers, accounts, cards and transactions tables, the foreign keys and the indexes.
- Configure the back end by entering the database URL, user name and password in the Spring Boot configuration file.
- Build and start the back end with Maven. The embedded server starts and the REST APIs become available.
- Install the front-end dependencies with npm and start the React application. The application opens in the browser and shows the Login screen.
- Verify the deployment by calling one API from Postman and by opening the Dashboard in the browser.

9.2 Environment Configuration

Configuration values that differ between machines are kept outside the code. The database host, port, schema name, user name and password are set in the Spring Boot configuration file, and the address of the back end used by the React application is set in the front-end environment configuration. This allows the same code to run on both team members' computers and on a different server without modification.

Setting	Where	Purpose
Database URL, user, password	Spring Boot configuration file	Connection of the repository layer to MySQL
Server port	Spring Boot configuration file	Port on which the REST APIs listen
Back-end base URL	React environment configuration	Address to which the front end sends requests
Isolation level / pool size	Spring Boot configuration file	Transaction and connection behaviour

Passwords and other secrets should not be committed to GitHub in a real deployment; they should be supplied through environment variables or a protected configuration file.

9.3 Hosting of Frontend and Backend

In the present version, the front end, the back end and the database all run on the local machine, which is sufficient for a simulation and costs nothing. The architecture allows each part to be hosted separately in the future: the React application can be built into static files and served by any web server, the Spring Boot application can be packaged as a single executable archive and run on a server, and the MySQL database can be moved to a dedicated host and extended with replication.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Building a banking simulation brings several technical challenges that are central to the course subject. The main issues that the design had to address are described below.

Data consistency. In a design where customer, account and transaction details are stored together, the same data is repeated and can become inconsistent. Deciding how to split the data into tables so that every fact appears once, and checking each table against the BCNF condition, required careful analysis of the functional dependencies.

Concurrency. When two operations on the same account arrive at the same time, both can read the same balance and the later write can overwrite the earlier one, which is known as a lost update. A transfer between two accounts adds a further risk, because two transfers in opposite directions can wait for each other, causing a deadlock.

Partial failure. A transfer consists of several steps. If a failure occurs after the debit but before the credit, money disappears from the system. Handling this correctly requires the steps to be grouped into one transaction with a proper transaction boundary in the service layer.

Query speed. A history query that scans the entire transactions table becomes slower as rows are added. It was necessary to find out which columns are used in search conditions and to confirm with EXPLAIN that the indexes are really used.

Integration. The React front end, the Spring Boot back end and the MySQL database must agree on data formats, status codes and error messages. Differences between the three caused mismatches that had to be found by testing each API in Postman before connecting the interface.

10.2 Solutions Applied

Issue	Solution applied
Inconsistent and redundant data	BCNF-normalised schema with primary and foreign keys
Lost updates	Row-level locking and a suitable isolation level for balance updates
Deadlocks between transfers	Locking the account rows in a fixed order
Partial failure of a transfer	Single ACID transaction with rollback on any error
Slow history and search queries	Secondary indexes verified with EXPLAIN
Layer mismatches	Layered architecture, standard status codes and Postman test collections

10.3 Current Limitations or Known Bugs

- The system runs on a single database server, so it is both a performance limit and a single point of failure. Replication and sharding are studied as concepts but are not yet deployed.
- Login security based on JWT is not yet implemented.
- The project is a simulation and is not connected to a real payment network or to other banks.
- Features such as loans, interest calculation and statements in downloadable form are outside the present scope.
- Performance has been studied on the data volumes available in the simulation; behaviour at very large scale has not been measured.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

JWT login security. The first planned enhancement is token-based authentication using JSON Web Tokens. At login the server will verify the credentials and issue a signed token, and every later request will carry the token so that the identity of the customer is verified on each call. This will strengthen the existing ownership validation and is the most important step towards a production-grade system.

Real replication. In the present version a single MySQL server holds all data. In the planned version, one primary server will accept writes and one or more replicas will copy its data. Read-heavy operations such as viewing the history can be served from the replicas, reducing the load on the primary, and a replica can take over if the primary fails, improving availability.

Sharding. As the number of customers and transactions grows, the data may no longer fit on one server. Sharding splits the data across several servers according to a shard key, for example the customer or account identifier, so that each server handles a part of the data. The design must ensure that a transfer between accounts held on different shards remains atomic, which requires distributed-transaction techniques.

11.2 Possible Integrations or Optimizations

- Role-based access for customers and bank staff on top of JWT.
- Scheduled and recurring transfers, and notification of transactions by email or message.
- Statements in downloadable form and spending summaries on the dashboard.
- Caching of frequently read data such as dashboard summaries.
- Further performance tuning with additional indexes, partitioning of the transactions table and monitoring of slow queries.
- Containerisation and cloud hosting with automated build and deployment.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Online Banking System Simulation is a full-stack banking application for accounts, deposits, withdrawals, transfers and transaction history. It uses React.js for the user interface, Spring Boot with REST APIs for the back end and MySQL for the database, and it is built around three database engineering ideas: a BCNF-normalised schema, ACID transactions and indexing.

The project began from the observation that simple banking systems have inconsistent data, concurrency errors and slow queries. By designing the schema carefully, by making every money movement a transaction and by supporting the main queries with indexes, the project addresses each of these problems. The layered architecture, with controller, validation, service and repository, keeps the code understandable and the rules in one place.

12.2 What Was Achieved

- A working banking simulation with reliable transactions, which meets all three objectives of the project.
- A BCNF-normalised schema of four tables: customers, accounts, cards and transactions.
- Deposits, withdrawals and transfers executed as ACID transactions.
- Indexing of frequently searched columns, analysed with EXPLAIN.
- A React interface with Login, Dashboard, Transfer, Transaction History, Cards and Profile screens.
- REST APIs tested in Postman, with balances remaining consistent after transfers.
- A study of replication and sharding as ways of scaling the system.

12.3 Skills Learned During Development

The project gave the team practical experience in several areas. In database design, we learned to identify functional dependencies, to normalise a schema to BCNF and to express it as an ER diagram. In transaction processing, we learned how atomicity, consistency, isolation and durability are obtained in practice, how isolation levels and locks relate to concurrency problems such as lost updates and deadlocks, and why a transaction boundary belongs in the service layer.

In performance engineering, we learned how to read a query plan with EXPLAIN and how an index changes the cost of a query. In back-end development, we learned to build a layered REST API with Spring Boot and to test it systematically with Postman. In front-end development, we gained experience in building component-based screens with React.js. Finally, we practised teamwork and version control with Git and GitHub, and we improved our ability to document and present a technical project.

CHAPTER -13 REFERENCES

Books and Papers

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Walls, C. (2016). Spring Boot in Action. Manning Publications.

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387.

Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. ACM Computing Surveys, 15(4), 287-317.

Documentation and Online Resources

Oracle Corporation. MySQL 8.0 Reference Manual (InnoDB, Transactions, Indexes, EXPLAIN, Replication). Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>

VMware / Spring Project. Spring Boot Reference Documentation. Retrieved from <https://docs.spring.io/spring-boot/>

Meta Open Source. React Documentation. Retrieved from <https://react.dev>

Postman Inc. Postman Learning Center. Retrieved from <https://learning.postman.com>

Git SCM. Git Documentation. Retrieved from <https://git-scm.com/doc>

GitHub Inc. GitHub Docs. Retrieved from <https://docs.github.com>

CHAPTER -14 APPENDICES

Screenshots of the app

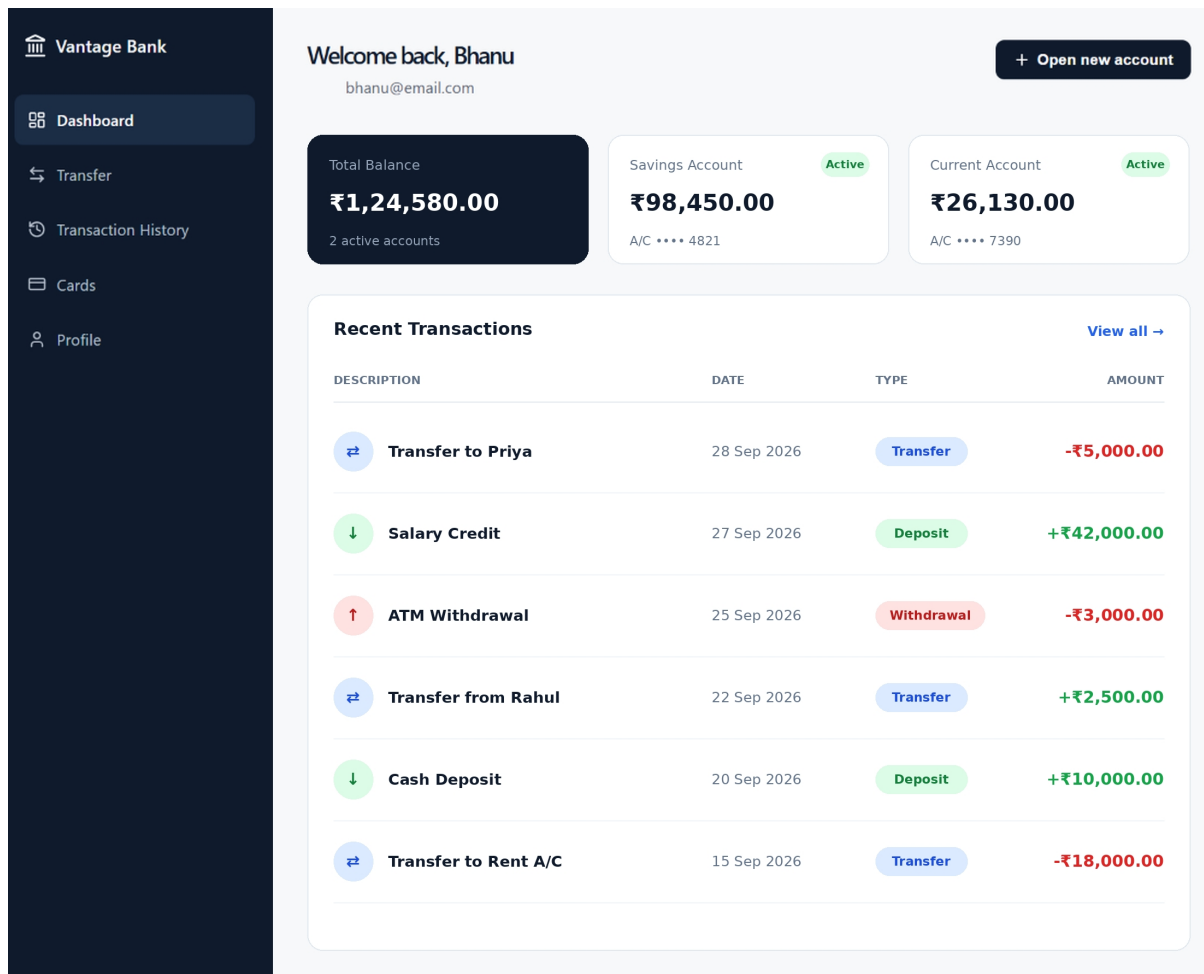


Figure A.1: Vantage Bank dashboard showing account balances and recent transactions

The dashboard displays a welcome message, the total balance of the customer, the savings and current accounts with their status and masked account numbers, and a list of recent transactions classified as Transfer, Deposit or Withdrawal. The side menu gives access to Dashboard, Transfer, Transaction History, Cards and Profile.

Sample code snippets

The following SQL, derived from the ER design in Chapter 5, creates the four tables and the main indexes.

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name  VARCHAR(50) NOT NULL,  
  last_name   VARCHAR(50) NOT NULL,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  phone       VARCHAR(15),  
  address     VARCHAR(255),  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE accounts (  
  account_id  INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  account_type VARCHAR(20) NOT NULL,  
  balance     DECIMAL(15,2) NOT NULL DEFAULT 0,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);  
  
CREATE TABLE cards (  
  card_id      INT AUTO_INCREMENT PRIMARY KEY,  
  account_id   INT NOT NULL,  
  card_number_masked VARCHAR(25) NOT NULL,  
  card_type    VARCHAR(20),  
  expiry_date  DATE,  
  status       VARCHAR(15),  
  FOREIGN KEY (account_id) REFERENCES accounts(account_id)  
);  
  
CREATE TABLE transactions (  
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,  
  account_id     INT NOT NULL,  
  related_account_id INT,  
  transaction_type VARCHAR(20) NOT NULL,  
  amount          DECIMAL(15,2) NOT NULL,  
  description      VARCHAR(255),  
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (account_id) REFERENCES accounts(account_id),  
  FOREIGN KEY (related_account_id) REFERENCES accounts(account_id)  
);  
  
CREATE INDEX idx_txn_account_time ON transactions (account_id, created_at);  
CREATE INDEX idx_accounts_customer ON accounts (customer_id);  
  
EXPLAIN SELECT * FROM transactions  
WHERE account_id = 1 ORDER BY created_at DESC LIMIT 10;
```

The sample above is a representative outline of the schema and of the history query; the complete scripts and source code are available in the GitHub repository.

Installation/setup instructions

The Online Banking System Simulation can be installed on any computer that has the required software of Chapter 2. First, install JDK 17 or higher, Maven, Node.js and MySQL Server with MySQL Workbench. Next, clone the project from GitHub and run the SQL script to create the database and tables. Then enter the database connection details in the Spring Boot configuration file and start the back end with Maven. Finally, install the front-end packages with npm and start the React application, which opens in the browser.

User manual or guide

When the application opens, the customer first reaches the Login screen. After logging in, the Dashboard shows the welcome message, the total balance, the accounts and the recent transactions. To open another account, select Open new account. To move money, choose Transfer from the side menu, select the account to debit, enter the destination account and the amount, and submit; the new balances appear on the Dashboard. To review past activity, open Transaction History. The Cards screen lists the cards linked to the accounts, and the Profile screen shows the customer's details.